

MarginGuard: o motor calcula, a IA explica

Arquitetura, fluxo de execução, ferramentas determinísticas e governança do agente de diagnóstico de margem da Vértice Retail.

O QUE ESTE DOCUMENTO DESCREVE

O sistema que responde perguntas de margem sobre a base da Vértice Retail sem deixar o modelo de linguagem calcular nada. Todo número citado numa resposta veio de uma função Python executada sobre os CSVs; o modelo escolhe a função e redige o texto.

COMO FOI GERADO

Os números deste documento são lidos do sistema no momento da geração: 39 ferramentas vêm do registro do motor, a contagem de testes vem dos arquivos de teste, e o exemplo da seção 15 é uma execução real do roteador e do motor. Reproduzível com `python -m scripts.gerar_documentacao`.

1. Visão geral

Problema de negócio. A Vértice Retail cresceu receita sem que a margem de contribuição acompanhasse. O diagnóstico precisa dizer onde a margem se perde, quanto vale cada frente e o que é recuperável — separando o que é fato apurado do que é estimativa sob premissa.

Objetivo do agente. Responder perguntas de margem com número rastreável, recorte declarado e força de evidência explícita, de forma que um comitê possa decidir sobre a resposta sem refazer a conta.

Papel da IA. Interpretar a pergunta, escolher a ferramenta e redigir. A IA não calcula, não estima e não preenche lacuna de dado. Quando a pergunta não se sustenta nos dados, a resposta é uma recusa medida, com a lacuna nomeada.

Escopo. Margem, desconto, frete, devolução, cancelamento, pendência, atendimento e sazonalidade, sobre os cinco CSVs do case. Fora de escopo: qualquer afirmação sobre cliente individual, LTV, CAC e churn — os campos de cliente não reconciliam com a transação (premissa P8).

2. Arquitetura

Seis camadas, com uma regra atravessando todas: cálculo é Python, texto é modelo.

```
Usuário
↓
Interface — painel web (frontend/) ou CLI (vertice/cli.py)
↓
Router determinístico — vertice/agent/router.py
↓
Agente ReAct — vertice/agent/react.py
↓
Ferramentas — 39 funções registradas em vertice/engine/
↓
Engine — pandas/scipy sobre data/*.csv
↓
Verificação numérica — vertice/agent/verificacao.py
```

↓
Trace — vertice/agent/trace.py
↓
Resposta

O motor (vertice/engine/) não importa nada do agente. É usável sozinho, sem chave de API e sem rede.

3. Fluxo de execução

O que acontece entre a pergunta e a resposta:

- 1. Roteamento.** O roteador determinístico classifica a intenção e extrai os parâmetros por regra em Python, antes de o modelo opinar. Se a rota for determinada, a ferramenta é executada direto.
- 2. Investigação.** Em pergunta aberta, o agente entra no laço ReAct: propõe uma ação, o motor executa, o resultado volta como observação, e o ciclo repete até haver evidência suficiente ou até o teto de iterações.
- 3. Síntese.** O modelo redige a resposta final no contrato de quatro seções, usando apenas os números que as ferramentas devolveram.
- 4. Verificação.** Todo número do texto é conferido contra o conjunto de números que saiu das ferramentas. Número sem lastro bloqueia a publicação.
- 5. Trace.** A execução inteira — pergunta, ações, parâmetros, resultados e resposta — é gravada em JSON e fica consultável pela API.

4. Router determinístico

O roteador existe para tirar do modelo a decisão de *o que perguntar aos dados*. Ele devolve um objeto `Roteamento` inspecionável, não um texto para outra regex interpretar.

Campo	O que carrega
intencao	a classe da pergunta (ex.: APLICAR_DESCONTO)
ferramenta	qual função do motor executar, ou None
parametros	canal, categoria, percentual, período já extraídos
confianca	quanto a regra casou, de 0 a 1
faltando	parâmetros que a pergunta não trouxe
motivo	por que esta rota foi escolhida — texto legível

Quando falta parâmetro obrigatório, a rota não é considerada determinada e o agente pergunta em vez de assumir um valor. É o que impede o sistema de inventar um teto de desconto que ninguém pediu.

5. Agente ReAct

Usado quando a pergunta é aberta — "por que a margem caiu?", "qual a prioridade?" — e o roteador não consegue determinar uma única ferramenta.

O protocolo é de texto: o modelo responde `AÇÃO:` e `PARÂMETROS:`, o parser lê, o motor executa e devolve a observação. O laço tem teto de iterações e fallbacks para resposta malformada, ferramenta inexistente e chamada repetida.

6. Ferramentas

39 ferramentas registradas por decorador em `vertice/engine/registry.py`. Cada uma declara prompt, schema e parâmetros, e devolve o recorte aplicado, a premissa e a marca de confiança (● fato calculado · ● estimativa sob premissa · ○ direção sem valor).

RENTABILIDADE E MARGEM · 4 FERRAMENTAS

Ferramenta	O que faz
<code>comparar_periodos</code>	Compara margem entre dois períodos (formato AAAA-MM ou AAAA-MM:AAAA-MM) e decompõe a variação de margem% em contribuição de desconto, custo e frete.
<code>margem_consolidada</code>	Margem de contribuição consolidada de toda a base aprovada: receita bruta, desconto, custo de produto, frete, margem em R\$ e %, e a decomposição MECE...
<code>margem_na_janela</code>	Margem da janela mais recente da base (padrão 7 dias) comparada com a janela imediatamente anterior de mesmo tamanho.
<code>margem_por_dimensao</code>	Margem aberta por uma dimensão, ordenada.

DESCONTOS E SIMULAÇÃO DE TETO · 5 FERRAMENTAS

Ferramenta	O que faz
<code>cenarios_reducao_desconto</code>	Cenários conservador / base / agressivo de redução do desconto no grupo que hoje recebe acima de um limiar (padrão 25%).
<code>simular_desconto_em_segmento</code>	Simula APLICAR um desconto hipotético a um recorte da base (canal, mês e/ou categoria) e devolve o efeito na margem do segmento e na margem CONSOLIDADA.
<code>simular_reajuste_preco</code>	Simula um REAJUSTE DE PREÇO em um recorte (canal, categoria e/ou mês) e devolve o efeito na margem do segmento e na consolidada.
<code>simular_teto_desconto</code>	Simula o efeito de impor um TETO de desconto: todos os pedidos com desconto acima do teto passam a receber exatamente o teto.
<code>tabela_faixas_desconto</code>	Itens por pedido, ticket médio e margem por faixa de desconto (0%, 0-10%, 10-20%, 20-25%, 25-30%, 30%+).

POLÍTICA DE DESCONTO E ALÇADAS · 2 FERRAMENTAS

Ferramenta	O que faz
<code>consultar_politica_desconto</code>	Diz se um desconto está dentro da política e QUEM precisa aprovar.
<code>pedidos_acima_do_teto</code>	Conta os pedidos que excedem o teto e mede dois sinais de contorno da política: concentração logo ABAIXO do teto (desconto fatiado para não pedir...

FRETE E ASSIMETRIA POR CANAL · 4 FERRAMENTAS

Ferramenta	O que faz
<code>custo_assimetria_frete</code>	Quantifica quanto um canal paga em frete que NÃO pagaria se seguisse a mesma regra dos demais canais.
<code>dispersao_componentes_entre_canais</code>	Coefficiente de variação (CV = desvio-padrão / média) de frete%, desconto% e custo% ENTRE os canais.
<code>politica_frete_por_canal</code>	Mapeia a política de frete efetiva de cada canal: % de pedidos com frete grátis, frete médio, frete como % da receita, e o cruzamento canal x faixa de...

regra_frete_por_limiar	Quanto um canal deixaria de pagar em frete se seguisse a MESMA REGRA OBJETIVA dos demais canais: a Vértice só arca com o frete de pedidos abaixo de um...
------------------------	---

PERDA DEPOIS DO PEDIDO · 2 FERRAMENTAS

Ferramenta	O que faz
pendencias_antigas	Pedidos com status 'Aguardando' há mais de N dias, e a margem não confirmada que eles carregam.
perda_pos_pedido	Ponte entre a margem CALCULADA e a margem que se REALIZA: quanto da margem de contribuição do ano se perde DEPOIS que o pedido é registrado, aberta nos...

DEVOLUÇÕES, ROAS E MARGEM NEGATIVA · 3 FERRAMENTAS

Ferramenta	O que faz
impacto_devolucoes	Vazamento de margem por devolução — o último componente do MECE (Receita – Desconto – Custo – Frete – Devolução).
pedidos_margem_negativa	Pedidos que destroem margem: margem_contribuicao < 0.
ranking_roas_canais	Ranking RELATIVO de eficiência de marketing por canal (mediana de ROAS e CAC).

SAZONALIDADE · 2 FERRAMENTAS

Ferramenta	O que faz
indice_sazonalidade	Índice sazonal por mês = volume do mês / volume médio mensal (>1 = mês acima da média).
tendencia_ajustada_sazonalidade	Tendência de uma métrica percentual ao longo dos meses com regressão linear sobre o tempo, informando se a inclinação é estatisticamente significativa.

VALIDAÇÃO ESTATÍSTICA · 5 FERRAMENTAS

Ferramenta	O que faz
anova_um_fator	ANOVA de um fator: testa se a média de uma métrica difere entre TODOS os níveis de uma coluna categórica.
classificar_evidencia	Classifica a força de uma evidência estatística (FORTE/MODERADA/FRACA) por critério objetivo: FORTE = $p < 0,001$ e ($R^2 > 0,7$ ou $n > 1000$); MODERADA = $p < 0,05$ e...
qui_quadrado_independencia	Qui-quadrado de independência entre duas colunas categóricas (ex: canal x frete_gratis, ou faixa_desconto x devolvido).
regressao_linear_multipla	Regressão linear múltipla (OLS) de uma métrica sobre variáveis explicativas numéricas.
teste_t_welch	Teste t de Welch (variâncias desiguais) comparando uma métrica entre dois grupos de uma coluna categórica.

AUDITORIA DE QUALIDADE DE DADO · 9 FERRAMENTAS

Ferramenta	O que faz
checar_completude	Percentual de valores nulos por coluna de uma tabela, ordenado do mais incompleto para o menos.
checar_devolucao_fora_prazo	Devoluções aceitas fora do prazo legal de arrependimento (CDC art.

<code>checar_duplicatas_chave</code>	Conta linhas cuja coluna-chave aparece mais de uma vez (ex: <code>order_id</code> repetido em vendas).
<code>checar_faixa_implausivel</code>	Valores fora de uma faixa esperada informada por quem conhece o negócio (ex: tempo de entrega negativo, quantidade zero, preço acima de um teto).
<code>checar_integridade_referencial</code>	Conta quantos valores de uma coluna (chave estrangeira) não têm correspondência na coluna-chave de outra tabela — ex: <code>sku_id</code> de vendas que não existe em...
<code>checar_outliers_iqr</code>	Registros fora de $Q1 - m \cdot IQR$ / $Q3 + m \cdot IQR$ numa coluna numérica (m = multiplicador, padrão 3.0 = outlier severo).
<code>coerencia_motivo_entrega</code>	Testa se o motivo de devolução 'Atraso na entrega' é coerente com o tempo de entrega efetivamente registrado no pedido.
<code>relatorio_auditoria_dados</code>	Roda de uma vez as checagens determinísticas de qualidade sobre as bases do case (integridade referencial, duplicata, completude, outlier, faixa...
<code>verificar_consistencia_categorica</code>	Tabela de contingência entre duas colunas categóricas, com contagem e percentual por célula — ex: categoria x motivo_devolucao, que revela combinação sem...

COBERTURA DE DADOS · 1 FERRAMENTAS

Ferramenta	O que faz
<code>cobertura_de_dados</code>	Responde se uma pergunta é sustentável pelos dados ANTES de tentar respondê-la.

KPIS DE ACOMPANHAMENTO · 2 FERRAMENTAS

Ferramenta	O que faz
<code>caso_base_do_plano</code>	Caso-base do plano: soma as alavancas com valor defensável e separa o que está sob controle INTERNO do que depende de terceiros.
<code>kpis_do_teto</code>	KPIs de aceite de um teto de desconto, para o roadmap e o teste controlado: taxa de desconto atual e esperada sob o teto (em % da receita bruta), regra de...

7. Motor determinístico

Todo cálculo financeiro e estatístico é Python sobre pandas e scipy: margem, decomposição MECE, simulação de teto, regra de frete, ponte do pós-pedido, teste t de Welch, qui-quadrado, regressão e sazonalidade.

Por que não no prompt. Um modelo de linguagem não é reproduzível entre execuções, não declara a premissa que usou e não tem como ser auditado número a número. Os 457 testes deste projeto verificam funções Python; não haveria como testar aritmética feita dentro de um prompt.

8. Verificação numérica

`verificacao.py` coleta todos os números que as ferramentas devolveram na execução e confere cada número do texto final contra esse conjunto. Número que não aparece em nenhum resultado bloqueia a publicação da resposta.

O hash da base fica fora do conjunto rastreável de propósito: um hash hexadecimal contém dígitos, e deixá-lo entrar daria lastro falso a qualquer número que casasse com um pedaço dele.

9. Trace e auditoria

Cada execução grava um JSON em `traces/` com pergunta, cada ação, os parâmetros, o resultado bruto de cada ferramenta e a resposta final. A API expõe `GET /api/trace/{id}`, e o painel liga a resposta ao trace que a produziu.

10. Dados

Cinco CSVs em `data/`: vendas, clientes, estoque, marketing e atendimento. A janela vai de 01/01/2023 a 26/01/2024.

Achado na base	Consequência
11,73% da receita bruta nunca foi recebida	métricas financeiras usam só status Aprovado (P1)
346 clientes distintos em vendas contra 15.000 no cadastro	LTV, CAC e segmentação não se ligam à transação (P8)
marketing.receita_gerada com atribuição sobreposta	nunca usada para valor em R\$ (P2)
684 registros de "Tamanho errado" em categoria sem grade	motivo de devolução não é tratado como causa (P9)
margem de contribuição é anterior à devolução	devolução é vazamento adicional, não descontado (P7)

11. Premissas

As 12 premissas estão numeradas em `PREMISSAS.md` e são citadas pelo código nos pontos onde valem.

ID	Premissa
P1	Só <code>`status_pagamento == 'Aprovado'`</code> é receita real
P2	<code>`marketing.receita_gerada`</code> não é usada para nenhum valor em R\$
P3	A identidade contábil da margem é sanidade, não causa raiz
P4	Desconto x volume é correlação, não causalidade
P5	Toda simulação assume volume constante
P6	Sazonalidade e mês parcial
P7	<code>`margem_contribuicao`</code> é anterior à devolução
P8	Margem negativa não é outlier a limpar
P9	Uma linha vazia descartada
P10	Encoding
P11	O recorte da simulação é parâmetro declarado, nunca filtro implícito
P12	Margem tem dois denominadores, e os dois viajam juntos

12. Limitações

O que o sistema **não** afirma, e por quê:

Elasticidade de desconto. Os dados são observacionais, sem grupo de controle nem variação exógena. Toda

simulação de teto assume volume constante e diz isso no retorno.

Meta em reais para o pós-pedido. A base não registra destino do item devolvido, custo da logística reversa nem motivo validado. A perda é dimensionada, não convertida em meta.

Teste A/B por cliente. O cálculo de poder mostra que medir perda de volume exigiria 887.164 clientes; a base tem 331. A inviabilidade é medida pelo próprio instrumento, não suposta.

Cliente individual. Fora de escopo por P8.

13. Governança

Política de desconto. Proposta com estado persistido, dono e data em cada transição. O fluxo vai de Rascunho a Simulação e congela: aprovar de verdade exige identidade autenticada, que é decisão de infraestrutura. O código da aprovação existe e é testado, incluindo a regra de que quem propõe não aprova.

Limites do agente. O agente lê; não escreve na base. Nenhuma rota grava em `data/*`.csv.

Tratamento de incerteza. Toda resposta declara força de evidência (FORTE, MODERADA ou FRACA) com o critério estatístico que a sustenta e a principal limitação.

Rastreabilidade. Trace por execução, hash da base em todo resultado e recorte declarado em todo valor em reais.

14. Testes

457 funções de teste, que geram 925 casos com parametrização. A suíte roda sem rede e sem chave de API.

Arquivo	Funções
test_agente.py	119
test_api.py	46
test_motor.py	36
test_router.py	33
test_auditoria.py	29
test_perguntas_negocio.py	29
test_relatorio_e_card.py	29
test_verificacao.py	29
test_politica.py	25
test_aceite_livro_de_numeros.py	22
test_experimento.py	16
test_filtros.py	12
test_prompts.py	12
test_integridade_dados.py	9
test_config.py	7
test_roteamento_do_deck.py	4

Testes de aceite. `test_aceite_livro_de_numeros.py` confere, ID a ID, se o motor reproduz os números publicados no Livro de Números — o teto de 20%, a regra de frete, a ponte do pós-pedido e os KPIs do roadmap. Se um falhar, o motor divergiu da apresentação, e isso é decisão humana, não ajuste de teste.

15. Exemplo ponta a ponta

Execução real, gerada no momento em que este PDF foi produzido.

PERGUNTA

O que acontece se aplicarmos 17% de desconto em Moda no Email Marketing?

ROTEADOR

Campo	Valor
Intenção	APLICAR_DESCONTO
Ferramenta	simular_desconto_em_segmento
Parâmetros	{ 'desconto_pct': 17.0, 'canal': 'Email Marketing', 'categoria': 'Moda' }
Confiança	0.95
Motivo	verbo de aplicação (1 gatilho(s)) + percentual explícito

RESULTADO DO MOTOR

Campo	Valor
Pedidos no segmento	843
% dos pedidos	3.45%
Receita bruta do segmento	R\$ 604.612,32
Desconto atual no segmento	7.17%
Desconto atual em R\$	R\$ 43.339,85
Desconto simulado a 17%	R\$ 102.784,09
Variação	R\$ 59.444,24
Segmento	{ 'canal': 'Email Marketing', 'categoria': 'Moda' }
Período coberto	{ 'inicio': '2023-01-01', 'fim': '2024-01-26', 'meses': 13, 'nota': 'valores acumulados dos 13 meses do recorte, NÃO mensais; para média por mês, divida por 13' }
Marca	●
Hash da base	0199d2f3e060

Leitura: aplicar 17% neste segmento **aumentaria** o desconto concedido em R\$ 59.444,24, porque o desconto que já se pratica ali é de 7.17%. O número é cenário sob premissa de volume constante, não previsão — e é a própria ferramenta que declara isso no

campo de premissa.

16. Estrutura do projeto

```
vertice-retail/
├─ data/ cinco CSVs do case (entrada; nunca reescritos)
├─ vertice/ motor + agente + API
│ └─ engine/ 39 ferramentas determinísticas, sem LLM
│   └─ agent/ router, ReAct, verificação, trace, prompts
├─ frontend/ painel web estático (sem framework, sem build)
├─ tests/ 457 funções de teste
├─ scripts/ auditoria das 95 perguntas e geração deste PDF
├─ design-system/ tokens de marca (procedência do visual)
├─ docs/ como rodar e relatório das 95 perguntas
├─ documentacao/ este documento
├─ README.md arquitetura e decisões
└─ PREMISSAS.md as 12 premissas, numeradas
```

17. Execução

Esta é a sequência exata usada para construir e verificar o projeto, na máquina em que ele foi desenvolvido. Os comandos estão na ordem em que foram executados.

AMBIENTE DE REFERÊNCIA

Item	Versão usada
Sistema operacional	Ubuntu 22.04.2 LTS
Python	CPython 3.10.12
Isolamento	venv (python3 -m venv venv)
pandas	2.3.3
numpy	2.2.6
scipy	1.15.3
openai	3.14.1
fastapi	0.141.1
uvicorn	0.53.0
httpx	0.28.1
pytest	9.1.1
pymupdf	1.28.2

O projeto roda em Windows, macOS e Linux. O ambiente acima é o que foi efetivamente usado — os comandos a seguir são os do WSL, e a seção seguinte traz o equivalente em PowerShell.

1. PRÉ-REQUISITO DO SISTEMA (UMA VEZ)

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
```

2. ENTRAR NO PROJETO

```
cd /mnt/c/caminho/para/vertice-retail
```

No WSL, o disco do Windows fica sob `/mnt/c/`. Todos os comandos partem da raiz do repositório — a pasta que contém `requirements.txt`.

3. CRIAR E ATIVAR O AMBIENTE VIRTUAL

```
python3 -m venv venv
source venv/bin/activate
```

4. INSTALAR AS DEPENDÊNCIAS

```
pip install -r requirements.txt
```

5. VERIFICAR — SEM REDE E SEM CHAVE DE API

```
python -m pytest
```

Esperado: **925 passando, 11 pulados**. Este passo não usa internet: o motor lê os CSVs e calcula em Python. É a verificação que precede qualquer demonstração.

```
python -m pytest tests/test_aceite_livro_de_numeros.py -v
```

Os 35 testes de aceite conferem, ID a ID, se o motor reproduz os números publicados. É o "Passo 0" da demonstração.

6. USAR SÓ O MOTOR, SEM IA

```
python -c "from vertice.engine import executar; import json; \
print(json.dumps(executar('simular_teto_desconto', {'teto_pct': 20}), \
ensure_ascii=False, indent=2))"
```

7. AGENTE SEM CHAVE (MODO MOCK)

```
python -m vertice.cli --mock
```

O texto vem de roteiro fixo; os números são reais, porque o motor roda de verdade.

8. AGENTE COMPLETO (PRECISA DE CHAVE)

```
export ELOAGENTS_API_KEY="sua-chave"
python -m vertice.cli --diagnostico # confere conexão e modelos
python -m vertice.cli # sessão interativa
```

9. PAINEL WEB

```
python -m vertice.api # http://localhost:8000
VERTICE_API MOCK=1 python -m vertice.api # sem chave nenhuma
```

10. REGERAR ESTE DOCUMENTO

```
python -m scripts.gerar_documentacao
```

EQUIVALENTE EM WINDOWS (POWERSHELL)

```
py -m venv venv
venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

```
python -m pytest
$env:ELOAGENTS_API_KEY = "sua-chave"
python -m vertice.api
```

Se o PowerShell recusar o script de ativação, rode uma vez: `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass`.

SE ALGO FALHAR

Sintoma	Causa e correção
ModuleNotFoundError: No module named 'vertice'	rodou fora da raiz do repositório, ou sem <code>python -m</code>
FileNotFoundError: data/vendas.csv	os cinco CSVs precisam estar em <code>data/</code>
ensurepip is not available	falta <code>python3-venv</code> : <code>sudo apt install python3-venv</code>
Um teste de aceite falhou	o motor divergiu do Livro de Números — investigue, não ajuste o teste
Tool choice is none, but model called a tool	o provedor recusou o protocolo em texto: use <code>VERTICE_TOOL_MODE=nativo</code>
HTTP 413 (pedido grande demais)	ligue <code>VERTICE_CATALOGO_COMPACTO=1</code>

Variáveis de ambiente, provedor alternativo e o detalhamento de cada comando: [docs/COMO_RODAR.md](#).